

Distributed Systems - 22_09

We'll use Reliable and Secure Distributed Programming as a book (recommended to buy).

The exam is divided in:

- written
 - theory questions
 - excersices
 - design of distributed: variation of something we've seen.
- oral: mandatory if you get 30.

Distributed Systems

✓ Definition 1

A distributed system is a set of spatially separate entity, each with a certain computational power, able to comunicate and coordinate to reach a common goal. Entities must be spatially separated.

✓ Definition 2

Another definition: a distributed systems is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

How to tollerate failure is a key point.

There are multiple examples in the everyday life that check the definition of distributed systems.

Examples

- WWW
- P2P network, when we download something (like, in Torrent) we use a DS
- ecc.

The common points between definitions are:

- we need a set of entities: computers, machines, software
- each entity must be distributed, separated spatially
- they have to have a common goal

We'll focus on static networks of processes, the model of the internet.

Let's introduce the concept of **local view**. We're a process and we're put into a network. We know basically nothing about the network, we only view as ourselves. How do we gain knowledge about the network? We need to receive messages. We need to know, for examples, if there is a cycle in the network. At the beginning we can't say. If our knowledge is only made by messages, then our vision of the world is delayed. We never know exactly what is happening right now. If the network is slow or unpredictable then solving problems becomes harder.

Let's see a simple problem.

≡ Example

We need to build fault-tolerant airplanes.

A key problem we need to study is the fault-tolerant agreement.

We have different processors (computers) which communicate via a bus. They need to decide, at a certain point, if they want to abort or commit a transaction.

If we have unbounded delays on the bus, or the system is asynchronous, solving the problem is impossible, even with a single failure.

Failures are a key-point of computer systems. There are several ways to tolerate them:

- failure detection: like a check-sum in a package
- failure masking: happens in TCP, if a message fails we send the message again
- failure tolerance: design a system to tolerate a failure in a transparent way
- failure recovery: when something doesn't work, we restart it (example).

The problem with failures is that they are uncertain. We don't know when someone or something will fail.

≡ Example

We have Bob & Alice. Located in two different hills. These two have to agree on a time to attack the enemy, either 10 or 11. The problem is that if they attack at the same time they win, else not. To communicate they need to send a soldier. The problem is that he can die, go away, ecc., so the message is not reliable.

An example: Bob decides to send a message to attack at 10, Alice receives it and agree. This is a simple way to decide something, one decides and the other receives.

At this point, as Bob, we cannot attack, as we are not aware if Alice has received the message. To fix this, Alice can send an ACK to say that she received the message. Let's see if the ACK is destroyed. As Alice we still cannot attack, because she has no confirmation that Bob has received the ACK. So to fix this Bob has to send another ACK. Bob can't still attack, he doesn't know that Alice know that he received an ACK.

There is no simple way to fix the problem this way.

This problem exists due to the uncertainty of the failure.

Even time does not help in solving the problem. In distributed systems the concept of **time** is important. The synchronization is not always perfect, especially in a large system. So we cannot use time.

In asynchronous systems we'll have to synchronize clock in a way that is acceptable enough, otherwise we'll not use time. There is no global clock. We'll have to use a concept of order of messages.

There will be a "fight" between asynchrony and failures. The only tool we have is the local view.

The methodology will be a formal one, with definition of a model, the use of abstraction, specification and algorithm.

Models, abstractions and basic concepts

✓ Definition

A **System** is a set of n processes in $\Pi : \{p_0, \dots, p_{n-1}\}$ with distinct identities. The number n is defined, so it's fixed.

All of the processes have a different identities (we use a number, like $p_0, \dots$).

The processes communicate using a communication graph $G : (\Pi, E)$. Usually G is complete, which means that between any pair of process there is a link. The communication happens by exchanging messages on communication links (edges of G).

A process is a state machine (possibly infinite, we can have infinite states). Let's have a program with a counter increasing (never overflows). If I increase the counter I have a machine with infinite states (each for each value of the counter). The process has a certain **state**, the internal memory and the content of the input and output buffers. We'll only talk about deterministic algorithm, no probability. The process communicate using Output Y and Input X with the above layer. Above there can be an human or another process. Regarding the buffers, we have InBuffer and OutBuffer, in InBuffer we receive messages that are then received by the process, in the OutBuffer the process puts messages that reach the link. The link is the "space" that transfers the message between processes. We'll use a function P_j to indicate a process.

$$P_j(q \in Q \cup Q_{in}, InBuf) = (q' \in Q, Send_msg \subset M)$$

$$OutBuf_j = OutBuf_j \cup Send_msg$$

$$InBuf_j = \emptyset$$

The internal states is the Q set.

When I send a message I go to a new state q' included in Q , and I create a new message in the M set.

The messages are inserted in the output buffers, thus being updated (second formula).

After the action, the InBuffer becomes empty, because i processed all the messages.

Model: asynchronous executions

An **Execution** is a sequence of **events**. An **event** is something that happens on my systems, changing the state.

There is an adversary that schedule a set of events (scheduler).

We have two kinds of events:

- Execution of local step of event: $Exec(i)$: process i executes one step of its state machine (going from state q to q').

- Delivery of a msg: $\text{Del}(m,i,j)$: Takes a message from a source process (its OutBuffer) to its InBuffer.

A **configuration** C_t at a certain time is a vector of components with each indicating the state of the process j .

Example:

$$C_0 = \langle (q_0, \{\}, \{\}), (q_1, \{m\}, \{\}), (q_2, \{\}, \{\}) \rangle$$

With three processes p_0, p_1, p_2 . It's a snapshot of the system. In p_1 we have, for example, a message m in InBuffer.

⚠ Warning

It's not possible that two processes start in the same state because, as we saw before, they have different identities (name is unique).

Another way to see execution is an infinite sequence that alternate configurations and events $(C_0, e_0, C_1, e_1, \dots)$.

see formulas on slides

Let's have an example, with C_0 stated as before. We cannot have a delivery event in this configuration, because each OutBuffer is empty. Let's go to configuration C_1 , where the message m is not in InBuffer anymore, while we have m' in OutBuffer in p_1 . So the event delivery can happen on p_1 , but we can also have the execution of local step.

We can have a graph, with each line indicating a process (p_0, p_1, p_2) . *see slide*

The execution stated is depicted by the diagram.

When we talk about delivery it means that the message arrives at destination.

Who decides the length of the arrows? It is decided of the adversary, an entity that controls the link and execution times of the process. An example in the graph is with e_2, e_9 and e_7, e_8 . The adversary decides that the message with e_7 arrives before e_2 .

We have to restrict the power of the adversary by using the concept of fair execution.

With fair execution it means that each process p_i executes an infinite number of local computations. We can have different intervals between each computations and anything can happen, but there must be another computation in the future. When we send a message, eventually there must be the event of receipt of the

message.

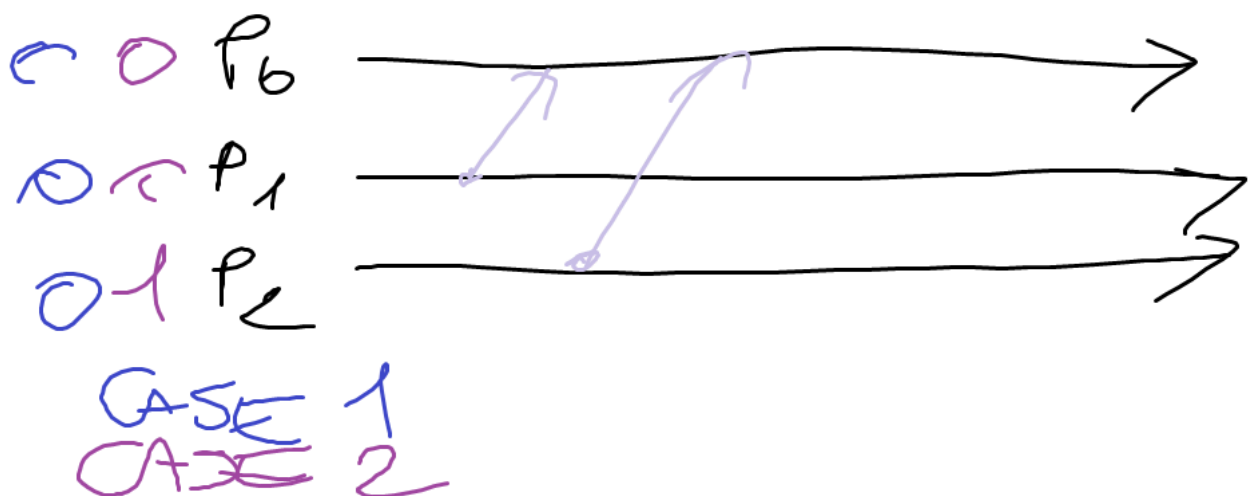
We have to consider fair execution because, in the other case, any non-trivial problem cannot be solved.

≡ Example

We have three processes p_0, p_1, p_2 .

We want to distinguish if there is a 1 in the input.

In case 1, p_0 only knows that he is zero. In order to know if there is a 1 in others, there has to be a delivery event. If the execution is unfair, there can be a case in which there is no receival from others.

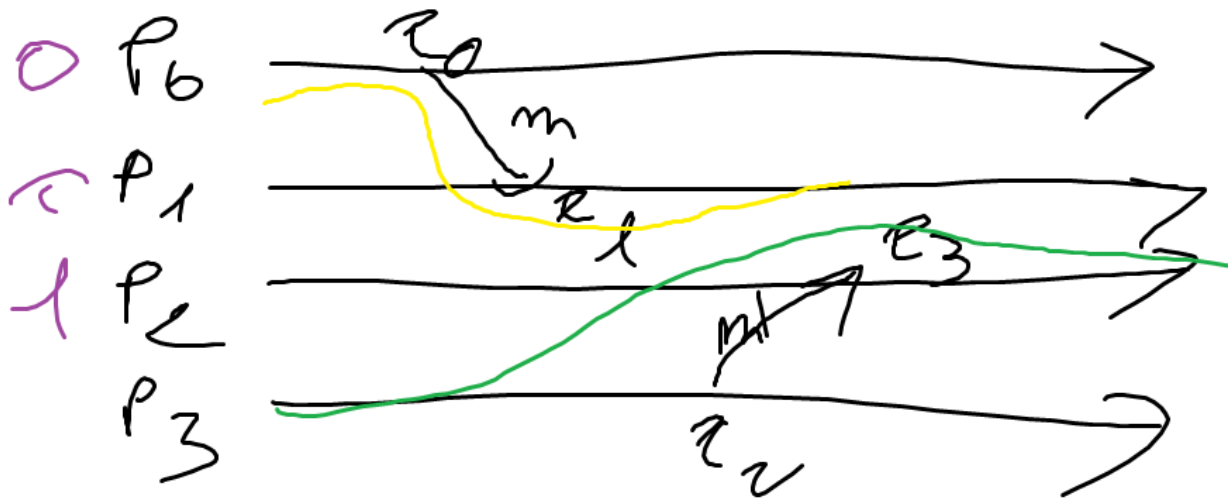


Given a certain execution E and a process p_j , we defined a **local execution** of process j $E|p_j$ the subset of events in E that impact p_j .

In the graph, for example, we have e_1 as a local execution (receive the message), but also e_0 , considering that the message was sent with e_0 , and I also know that, somehow, the message was sent. The same goes for p_2 , we know about e_3 but also e_2 .

For the previous example, we cant say that this algorithm with this graph solves the problem. In fact p_1 only knows about p_0 and p_1 , while p_2 only knows about p_2

and p_3 .



Let's take another example, one with one in the left (same as previous), and the right, with the order of the messages swapped (m' is sent first).

The local view is exactly the same, for p_1 we have the local view of e_1 and e_0 . Also with p_2 , we have e_3 and e_2 .

This is the concept of indistinguishability of execution. $\forall p_j \in \Pi, \mathbf{E}|_{p_j} = \mathbf{E}'|_{p_j}$.

✓ Theorem

In the asynchronous model, there is no distributed algorithm capable of reconstructing the system execution.

If, in the previous example, there was another execution (from p_2 to p_1), we would be able to distinguish the execution because of the ordering (like we have the execution happening before or after e_1).

A system is **Synchronous** when there is a fixed bound between:

- delay of the messages.
- execution time of actions executed by process, we assume in DS that this time is instantaneous.
- execution of actions.

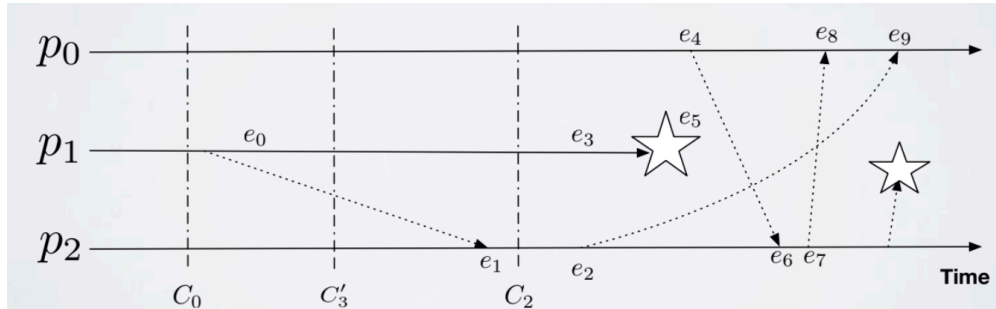
We then have the eventually-synchronous systems. It's a system which can have a period of synchronicity, which will be solved eventually.

Failures

We'll look at two models of failures:

- Crash-Stop Failure: a lot of blockchain systems uses this one.
- Byzantine Failure: an adversary, worst kind of failure.

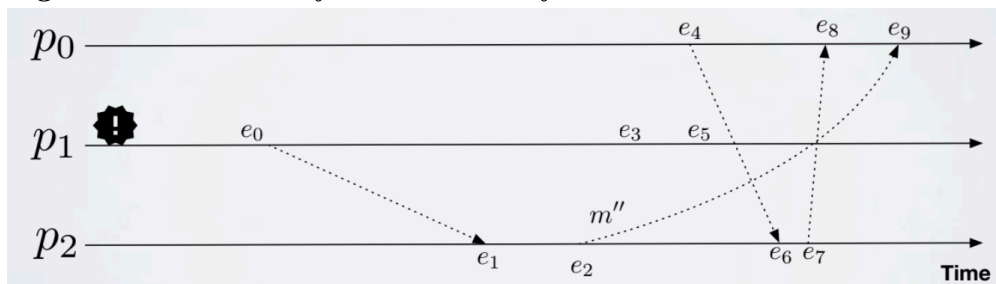
A Crash-Stop failure can be regarded as another event $Crash(p_j)$. When this event happens the process p_j does not execute any local computation step (no Exec(j)).



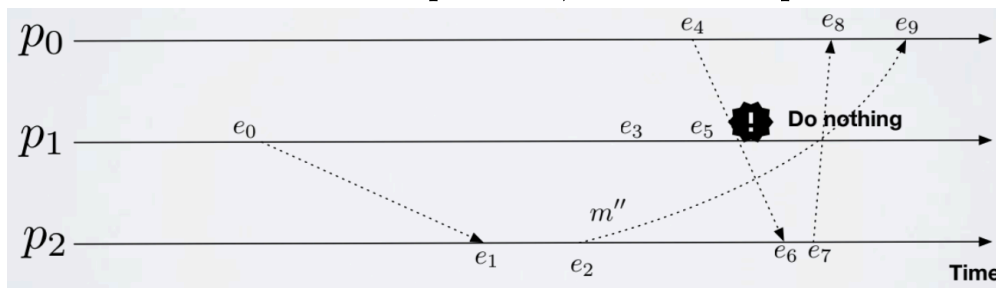
What happens if a message is sent to a crashed process? (the star indicates that something has crashed).

The byzantine failure will be also indicated by an event, with an exclamation mark.

When this event happen, the process does not stop working, but will behave in an arbitrary way, it does not follow the algorithm anymore, instead he will run an algorithm defined by the adversary.



Let's say we have an algorithm, it works in a system of 10 processes and can tolerate 1 byzantine process. We have to use this algorithm in a system with 10 processes and a crash-stop failure. We can still use it because a byzantine failure behaves also as a crash-stop failure, so it's not a problem.



We can say that $Crash \subset Byzantine$.

What if we have an algorithm with 10 processes and 1 crash-stop and we want to make it work in a system with 10 processes and 1 byzantine failure. In this case it won't work, the crash cannot act as a byzantine.

What is a **correct process**?

It's a process that does not experience a failure. The problem is the uncertainty. The threshold, indicated with f , is the maximum number of processes that can experience a failure event in a execution. We usually also bound f , like f cannot be more than $n - 1$, with $f \leq n$.